

Univerzitet u Sarajevu
Elektrotehnički Fakultet
Objektno Orijentisana Analiza i Dizajn

ETF Talent Program

Sedmi projektni zadatak
Strukturalni paterni

Maj 2026.

Sadržaj

1	Uvod i motivacija za primjenu strukturalnih paterna	2
2	Odabrani strukturalni paterni	2
2.1	Facade patern — Servisna fasada za studentske operacije	2
2.1.1	Problem bez paterna	2
2.1.2	Rješenje primjenom Facade paterna	3
2.1.3	Nove klase dodane u dijagram	3
2.2	Decorator patern — Proširivanje e-mail servisa	4
2.2.1	Problem bez paterna	4
2.2.2	Rješenje primjenom Decorator paterna	4
2.2.3	Nove klase dodane u dijagram	4
3	Ažurirani dijagram klasa	5
4	Pregled dodanih klasa	7

1 Uvod i motivacija za primjenu strukturalnih paterna

Sistem **ETF Talent Program** trenutno posjeduje dobro definisanu domensku hijerarhiju klasa i transakcijske klase. Međutim, analizom interakcija između klasa i kontrolera identifikovane su dvije ključne strukturalne slabosti koje komplikuju proširivost i održavanje sistema.

Prva slabost tiče se načina na koji kontroleri pristupaju domenskim klasama. Klase `StudentProfil`, `AkademskiPodatak`, `RangParametri` i `Verifikacija` čine složen podsistem studentskih podataka. Kontroleri poput `StudentProfilController` i `VerifikacijaController` direktno ovise o svim tim klasama, što narušava princip niskog sprežanja i otežava buduće promjene.

Druga slabost odnosi se na klasu `EmailNotifikacija`. Kako sistem raste, potrebe vezane za slanje e-pošte postaju sve složenije: treba dodati logiranje svakog pokušaja slanja, mehanizam automatskog ponovnog pokušaja u slučaju greške servisa, te eventualno enkripciju sadržaja ili provjeru formata. Dodavanje tih odgovornosti direktno u klasu `EmailNotifikacija` narušava *Single Responsibility Principle* i onemogućava fleksibilnu kombinaciju ponašanja.

Iz navedenih razloga, u nastavku su odabrana i obrazložena dva strukturalna paterna koji neposredno rješavaju ove probleme:

1. **Facade pattern** — za upravljanje složenim studentskim podsustavom
2. **Decorator pattern** — za fleksibilno proširivanje funkcionalnosti e-mail servisa

2 Odabrani strukturalni paterni

2.1 Facade pattern — Servisna fasada za studentske operacije

2.1.1 Problem bez paterna

Trenutno u sistemu, svaki kontroler koji treba obaviti operaciju vezanu za studenta direktno radi s više klasa istovremeno. Na primjer, da bi prikazao rang i profil studenta, `StudentProfilController` mora:

- Dohvatiti `StudentProfil` radi podataka o profilu,
- Dohvatiti listu `AkademskiPodatak` radi podataka o položenim predmetima,
- Pribaviti parametre iz `RangParametri` radi izračuna ranga,
- Provjeriti status u `Verifikacija` radi prikaza statusa verifikacije.

Ovaj pristup uzrokuje visok stepen sprežanja između kontrolera i domenskih klasa, što znači da svaka promjena u internoj logici tih klasa može zahtijevati izmjenu više kontrolera. Uz to, isti kompleksni redosljed poziva mora se ponavljati u više mjesta u kodu (npr. i u `VerifikacijaController`-u i u `PretragaController`-u).

2.1.2 Rješenje primjenom Facade paterna

Facade patern nudi jednostavan interfejs koji sakriva složenost podsistema iza jedne klase. Umjesto da kontroleri direktno komuniciraju s više domenskih klasa, oni komuniciraju isključivo s fasadom, a fasada interno koordinira sve pozive.

U kontekstu ETF Talent Programa, uvodi se interfejs `IStudentServis` koji deklarira sve operacije vezane za studentski podsistem. Klasa `StudentServisFasada` implementira taj interfejs i interno koristi `StudentProfil`, `AkademskiPodatak`, `RangParametri` i `Verifikacija`. Kontroleri sada ovise samo o apstraktnom interfejsu `IStudentServis`, što omogućava laku zamjenu implementacije bez ikakvih promjena na strani kontrolera (Dependency Inversion Principle).

Konkretno prednosti uvođenja fasade u sistem su:

- **Smanjenje sprežanja:** Kontroleri ne moraju znati ništa o internoj organizaciji studentskog podsistema.
- **Eliminacija duplikacije:** Logika koordinacije poziva više nije rasprostranjenja po više kontrolera.
- **Lakša testabilnost:** Interfejs `IStudentServis` može biti lako zamjenjen mock implementacijom u unit testovima.
- **Proširivost:** Dodavanje nove implementacije fasade (npr. za cache sloj) ne zahtijeva promjene u postojećem kodu kontrolera.

2.1.3 Nove klase dodane u dijagram

IStudentServis «interface»

Interfejs koji deklarira sve operacije namijenjene studentskom podsistemu. Metode:

- `getStudentProfil(id: Long) : StudentProfil`
- `izracunajRang(id: Long) : double`
- `verificirajPodatke(id: Long) : void`
- `getAkademskiPodaci(id: Long) : List`

StudentServisFasada

Konkretna implementacija interfejsa `IStudentServis`. Agregirane klase (atributi):

- `studentProfil : StudentProfil`
- `akademskiPodatak : AkademskiPodatak`
- `rangParametri : RangParametri`
- `verifikacija : Verifikacija`

Fasada koordinira pozive prema ovim klasama i vraća objedinjene rezultate kontrolerima.

Odnosi u dijagramu: `StudentServisFasada` implementira (zavisnost s isprekidanom strelicom i praznim trouglom) interfejs `IStudentServis`.

2.2 Decorator patern — Proširivanje e-mail servisa

2.2.1 Problem bez paterna

Klasa `EmailNotifikacija` trenutno sadrži atribut `brojPokusaja` i metodu `dodajURed()`, što ukazuje da je logika ponovnog slanja i čekanja *već* ugrađena direktno u klasu. Kako sistem evoluirá, bit će potrebno dodati:

- **Logiranje:** Svaki pokušaj slanja treba biti zabilježen u klasi `Log`,
- **Automatski retry:** Ponovni pokušaj slanja nakon greške s konfigurabilnim intervalima,
- **Validacija:** Provjera formata e-mail adrese prije slanja.

Direktno dodavanje svih tih odgovornosti u jednu klasu krši i *Single Responsibility Principle* i *Open/Closed Principle*. Svaka nova zahtjev zahtijeva izmjenu već testirane klase.

2.2.2 Rješenje primjenom Decorator paterna

Decorator patern omogućava dinamičko dodavanje odgovornosti objektima bez izmjene njihove klase. Uvodi se zajednički interfejs `ISmailServis` koji definišu operaciju slanja e-pošte. Konkretna implementacija (`SmtplibEmailServis`) obavlja osnovno slanje putem SMTP-a ili SendGrid API-ja. Apstraktni dekorator `EmailServisDecorator` implementira isti interfejs i agregiruje referencu na `ISmailServis`, čime se omogućava zamotavanje konkretnih implementacija. Konkretni dekoratori `LogirajuciEmailServis` i `RetryEmailServis` dodaju logiranje odnosno logiku ponovnog pokušaja *bez ikakve izmjene* `SmtplibEmailServis`-a.

Klasa `EmailNotifikacija` sada ne komunicira direktno s konkretnim servisima, već koristi apstrakciju `ISmailServis`. Na ovaj način dekoratori se mogu slobodno kombinovati i slagati: npr. `LogirajuciEmailServis` može zamotati `RetryEmailServis`, koji zamotava `SmtplibEmailServis`, čime se dobija e-mail servis koji i logira i automatski ponavlja pokušaj slanja.

Prednosti uvođenja Decorator paterna:

- **Otvoreno/zatvoreno:** Nove funkcionalnosti (dekoratori) dodaju se bez izmjene postojećih klasa.
- **Fleksibilna kombinacija:** Dekoratori se mogu slagati u bilo kom redosljed i kombinovati po potrebi.
- **Princip jedne odgovornosti:** Svaki dekorator odgovoran je za tačno jednu dodatnu odgovornost.
- **Dependency Inversion:** `EmailNotifikacija` ovisi o `ISmailServis` apstrakciji, a ne o konkretnoj implementaciji.

2.2.3 Nove klase dodane u dijagram

IEmailServis «interface»

Interfejs koji apstrahuje operaciju slanja e-pošte. Metoda:

- `posaljiEmail(n: EmailNotifikacija) : void`

SmtplibEmailServis

Konkretna implementacija IEmailServis. Komunicira s eksternim e-mail servisom (SMTP ili SendGrid). Atributi: `smtpHost`, `smtpPort`, `apiKey`.

EmailServisDecorator «abstract»

Apstraktni dekorator koji implementira IEmailServis i agreguje referencu `wrapped : IEmailServis`. Sve metode po defaultu delegiraju poziv zao-motanom objektu.

LogirajuciEmailServis

Nasljeđuje EmailServisDecorator. Prije i nakon svakog poziva `posaljiEmail()` bilježi akciju u Log (datum, status isporuke, e-mail primaoca).

RetryEmailServis

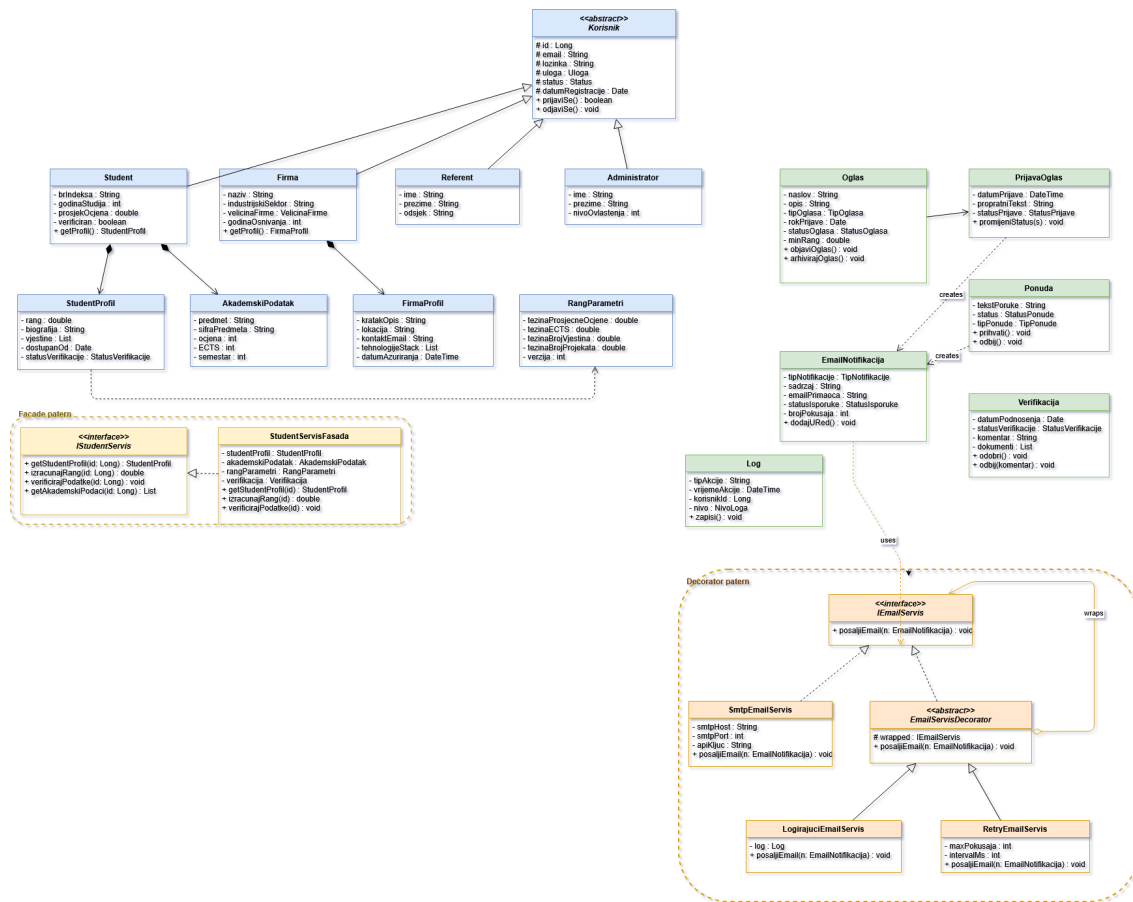
Nasljeđuje EmailServisDecorator. U slučaju greške pri slanju, automatski ponavlja pokušaj do `maxPokusaja` puta s pauzom `intervalMs` milisekundi između pokušaja.

Odnosi u dijagramu:

- SmtplibEmailServis i EmailServisDecorator implementiraju (isprekidana strelica s praznim trouglom) IEmailServis.
- EmailServisDecorator agreguje (prazni dijamant) IEmailServis.
- LogirajuciEmailServis i RetryEmailServis nasljeđuju (puna strelica s praznim trouglom) EmailServisDecorator.
- EmailNotifikacija koristi (isprekidana strelica «uses») IEmailServis.

3 Ažurirani dijagram klasa

Na sljedećoj slici prikazan je kompletni ažurirani dijagram klasa koji uključuje sve originalne klase sistema ETF Talent Program, dopunjene novim klasama koje uvode Facade i Decorator strukturalne paterne. Nove klase su vizualno istaknute žutom (Facade) i narandžastom (Decorator) bojom pozadine.



Slika 1: Ažurirani dijagram klasa sa dodanim strukturalnim paternima (Facade i Decorator)

4 Pregled dodanih klasa

Klasa / Interfejs	Patern	Uloga
IStudentServis	Facade	Interfejs fasade — deklarira sve operacije studentskog podsistema
StudentServisFasada	Facade	Konkretna fasada — koordinira StudentProfil, AkademskiPodatak, RangParametri, Verifikacija
IEmailServis	Decorator	Zajednički interfejs za sve e-mail implementacije i dekoratore
SmtplibEmailServis	Decorator	Osnovna konkretna komponenta — šalje e-poštu putem SMTP/SendGrid
EmailServisDecorator	Decorator	Apstraktni dekorator — agregiruje IEmailServis, delegira pozive
LogirajuciEmailServis	Decorator	Konkretni dekorator — bilježi svaki pokušaj slanja u Log
RetryEmailServis	Decorator	Konkretni dekorator — automatski ponavlja slanje pri grešci

Tablica 1: Pregled novih klasa dodanim primjenom strukturalnih paterna